

Reviewer Pack — Minimal Local Gromov-Witten Invariant and ACC0 Circuit Compl...

Entry 4b6a0dbb0f4e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 17:02:12 UTC

Minimal Local Gromov-Witten Invariant and ACC0 Circuit Complexity

Entry ID: 4b6a0dbb0f4e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 17:02:12 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry: Gromov-Witten Theory **Field B** (complexity object): Boolean Circuit Complexity: ACC⁰

Statement:

[‘For every explicit function f in P with an ACC⁰ circuit, there exists a tropical curve C such that the minimal local Gromov-Witten invariant of C is $\Theta(\log n)$.’, ‘The minimal local Gromov-Witten invariant for a given tropical curve C can be computed in subexponential time.’, ‘For all instances with $n \leq 40$, if f has an ACC⁰ circuit of size $s(n)$, then the minimal local Gromov-Witten invariant of its associated tropical curve is at least $\Theta(\log s(n))$.’]

Rationale (proposer’s reasoning):

[‘The Gromov-Witten invariants provide a rich algebraic-geometric structure that could potentially reveal hidden complexity-theoretic properties.’, ‘Local Gromov-Witten theory focuses on specific parts of the moduli space, which may allow for stronger lower bounds than global ones.’, ‘A connection between local Gromov-Witten invariants and ACC⁰ circuit complexity would provide a novel approach to proving lower bounds for explicit functions.’]

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4dfddad827fe73f1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal local Gromov-Witten invariant of the associated tropical curve is $\Theta(\log s(n))$ where $s(n)$ is the ACC⁰ circuit size.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Local Gromov-Witten Invariant AND ACC0 Circuit Complexity - Gromov-Witten Theory IN ALGEBRAIC GEOMETRY AND Boolean Circuit Complexity - Tropical Curve AND Gromov-Witten Invariant AND ACC⁰

Top relevant hits considered: - [http://arxiv.org/abs/alg-geom/9612009v1] The elliptic Gromov-Witten invariants of CP^3 - [http://arxiv.org/abs/2112.09180v2] A new approach to the operator formalism for Gromov-Witten invariants of the cap and tube - [http://arxiv.org/abs/1503.00460v3] Quantum Reidemeister torsion, open Gromov-Witten invariants and a spectral sequence of Oh - [http://arxiv.org/abs/1407.1370v5] Equivariant Gromov-Witten Invariants of Algebraic GKM Manifolds - [http://arxiv.org/abs/1701.07821v1] Gromov-Witten theory via Kuranishi structures - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/2510.17400v1] Tropical super Gromov-Witten invariants

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=124, elapsed=240.3s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_tropical_curve(f):
        n = int(math.log2(len(f)))
        curve = []
        for i in range(n):
            for j in range(i+1, n):
                if f[2**i + 2**j] == 1:
                    curve.append((i, j))
        return curve

    def min_local_gromov_witten_invariant(curve):
        if not curve:
```

```

        return 0
    return len(curve)

for n in [5, 10, 15, 20, 30, 40]:
    f = generate_boolean_function(n)
    curve = compute_tropical_curve(f)
    invariant = min_local_gromov_witten_invariant(curve)

    if invariant < math.log(len(f), 2):
        return {
            "metric_name": "min_local_gromov_witten_invariant",
            "metric_value": invariant,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"n={n}, s(n)={len(f)}, invariant={invariant}"
        }

    return {
        "metric_name": "min_local_gromov_witten_invariant",
        "metric_value": math.log(len(f), 2),
        "instances_tested": 6,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
        seeds = random.sample(primes, 30)

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"n={result['counterexample']}\", first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot confirm whether the conjecture’s support conditions are met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14865
2	preregistration	ollama_remote	glm4:latest	0	0	9412
3	novelty	ollama_remote	glm4:latest	0	0	8372
4	novelty	ollama_remote	glm4:latest	0	0	9888
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11398
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6859
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9555
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9233
9	judge	ollama_remote	glm4:latest	0	0	12238

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 91819 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/4b6a0dbb0f4e.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in [research/audit/4b6a0dbb0f4e.jsonl](#) for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: [research/pvsnp_sandbox/test_*.py](#) (per-cycle)

Replay tarball: [research/replay/4b6a0dbb0f4e.tar.gz](#) (if generated)